Homework 3

Pavlo Drizd – Matricola: `577440`
Email istituzionale: `pav.drizd@stud.uniroma3.it`
Corso di Ingegneria dei Dati – A.A. 2026/2027
Università degli Studi Roma Tre

Repository GitHub: https://github.com/pavlodrizd/IngegneriaDeiDati_HW3

1 Introduzione e Struttura del Progetto

L'obiettivo di questo homework è lo sviluppo di un sistema locale di indicizzazione e ricerca full-text basato su **Apache Lucene 10.6.0** (Java 21), progettato per operare su una directory di file `.txt` considerando due campi distinti: il nome del file (`nome`) e il testo al suo interno (`contenuto`).

Il progetto è stato mantenuto snello e modulare, suddividendo il lavoro in tre componenti principali:

- `generate_dataset.py`**:** script Python utilizzato per generare il dataset di valutazione di **10.000 file** `.txt` in `data/corpus/` e il file `data/ground_truth.tsv` necessario per il calcolo delle metriche;
- `Indexer.java`**:** classe che gestisce la configurazione degli analyzer tramite `PerFieldAnalyzerWrapper`, legge i file dalla directory locale e costruisce l'indice su disco (`FSDirectory`), misurando i tempi di indicizzazione al crescere del numero di documenti;
- `Searcher.java`**:** interfaccia a riga di comando che interpreta le query secondo la sintassi `<nome|contenuto> <termini o "frase">`, esegue la ricerca tramite `IndexSearcher` con funzione di ranking `BM25Similarity` e include la modalità `-test` per eseguire in automatico le 10 query di benchmark calcolando Precision, Recall e F1-Score.

2 Costruzione del Dataset e Definizione della Ground Truth

Per testare il sistema su una scala significativa e valutare concretamente sia i tempi di indicizzazione sia le metriche di efficacia, ho generato un corpus di **10.000 file** `.txt` divisi equamente in 5 categorie tematiche da 2.000 documenti ciascuna (`space`, `med`, `tech`, `sport`, `politics`). I testi di base sono stati estratti dal dataset pubblico *20 Newsgroups* tramite la libreria `scikit-learn` (categorie `sci.space`, `sci.med`, `comp.graphics`, `rec.sport.baseball` e `talk.politics.guns`).

2.1 Il Problema del Calcolo della Recall su 10.000 File

Valutare un sistema di Information Retrieval su 10.000 documenti reali pone subito un problema pratico: mentre per la *Precision* basterebbe controllare manualmente se i primi k documenti restituiti sono pertinenti, per calcolare la **Recall** ($R = TP/(TP + FN)$) è indispensabile conoscere a priori quanti sono in totale i documenti rilevanti presenti nell'intero corpus ($|R_q| = TP + FN$). Su 10.000 file non etichettati, verificare manualmente i falsi negativi per ogni query non è fattibile.

Per avere una *Ground Truth* oggettiva e calcolare le metriche in modo esatto, ho strutturato lo script di generazione dei dati nel modo seguente:

1. **Pulizia del testo di base:** durante l'estrazione degli articoli da *20 Newsgroups*, il testo viene normalizzato rimuovendo eventuali occorrenze sparse dei termini specifici scelti per i test, così da evitare match casuali non tracciati;

2. **Inserimento controllato di casi rilevanti e distrattori:** per ciascuna categoria tematica, in sottoinsiemi di dimensione nota (tra 40 e 80 documenti) ho assegnato nomi di file strutturati (es. `space_apollo11_lunar-module_00001.txt`, `tech_gpu_rtx4090_benchmark_04001.txt`) e inserito frasi pertinenti che presentano variazioni morfologiche reali (es. *vaccinations*, *vaccinated*, *vaccinating*) o aggettivi interposti (es. *"magnetic resonance functional imaging"*). Contemporaneamente, in altri file ho inserito dei **distrattori** (documenti non rilevanti per il bisogno informativo ma che condividono alcuni termini con le query, come file `rtx3080` o articoli su missili nucleari);

3. **Esportazione della Ground Truth:** lo script salva nel file `data/ground_truth.tsv` la lista esatto dei file rilevanti per ognuna delle 10 query, permettendo alla classe `Searcher` di confrontare i file restituiti da Lucene con quelli attesi.

3 Architettura dell'Indice e Scelta degli Analyzer

3.1 Rappresentazione dei Documenti e Campi

In Apache Lucene esiste una corrispondenza 1 : 1 tra ciascun campo (`Field`) e la propria *posting list* (dizionario dei termini e lista invertita delle occorrenze). Pertanto, definire due campi all'interno dei documenti equivale a creare due indici invertiti logici distinti gestiti all'interno della medesima `FSDirectory` (`target/idx`). Ogni file `.txt` viene convertito in un oggetto `Document` composto da:

- `nome` (`new TextField("nome", f.getName(), Field.Store.YES)`)**:** ho scelto `TextField` anziché `StringField` perché `StringField` indicizza l'intera stringa come un unico token esatto senza analisi. Usando `TextField`, invece, è possibile interrogare il nome del file per singole parole chiave o sequenze di termini. Ho impostato `Field.Store.YES` perché a tempo di ricerca è necessario leggere e stampare a video il nome del file associato ai `ScoreDoc` restituiti;

- `contenuto` (`new TextField("contenuto", contenuto, Field.Store.NO)`)**:** anche il testo interno è indicizzato come `TextField`, ma con `Field.Store.NO`. Come visto a lezione, questa scelta permette di indicizzare completamente i token, le frequenze e le posizioni nelle *posting list* (rendendo possibili sia il ranking `BM25` sia le `PhraseQuery`), evitando però di copiare l'intero testo dei 10.000 file nei *stored fields* dell'indice, con un notevole risparmio di spazio su disco e di I/O.

3.2 Configurazione degli Analyzer (`PerFieldAnalyzerWrapper`)

I campi `nome` e `contenuto` hanno caratteristiche lessicali molto diverse e richiedono due pipeline di analisi dedicate, combinate tramite `PerFieldAnalyzerWrapper`:

```
Analyzer nomeAnalyzer = CustomAnalyzer.builder()
        .withTokenizer("whitespace")
        .addTokenFilter("wordDelimiterGraph")
        .addTokenFilter("lowercase")
        .build();

Map<String, Analyzer> perField = new HashMap<>();
perField.put("nome", nomeAnalyzer);
perField.put("contenuto", new EnglishAnalyzer());

Analyzer analyzer = new PerFieldAnalyzerWrapper(new StandardAnalyzer(), perField);
```

1. **Analyzer per il campo `nome`:** i nomi dei file contengono quasi sempre separatori come underscore (`_`), trattini (`-`), punti per l'estensione (`.txt`) o sigle alfanumeriche attaccate (es. `rtx4090`, `apollo11`). Uno `StandardAnalyzer` non spezza i token separati da underscore né divide le transizioni tra lettere e numeri, mentre un `WhitespaceAnalyzer` lascerebbe l'intero nome del file come un unico token. Ho quindi costruito un `CustomAnalyzer` basato sul filtro

`wordDelimiterGraph` seguito da `lowercase`: in questo modo un file come `space_apollo11_lunar-module_00001.txt` viene suddiviso nei token `[space, apollo, 11, lunar, module, 00001, txt]` mantenendo le posizioni relative per le *phrase query*. Sui nomi dei file non ho applicato né rimozione di stopword né stemming, per evitare di alterare sigle tecniche o codici;

2. **Analyzer per il campo `contenuto`:** essendo il dataset basato su articoli in lingua inglese, ho scelto `EnglishAnalyzer`. Rispetto a uno `StandardAnalyzer`, `EnglishAnalyzer` aggiunge due passaggi fondamentali per il testo libero: la rimozione delle *stopword* inglesi (`StopFilter`, che scarta parole frequentissime e poco informative come `the`, `of`, `and`, `in`) e lo **stemming di Porter** (`PorterStemFilter`). Quest'ultimo riduce le parole alla loro radice morfologica (es. `vaccinations`, `vaccinated`, `vaccinating` → `vaccin`), permettendo di recuperare documenti pertinenti anche quando la query usa una forma verbale o plurale diversa da quella presente nel testo.

Lo stesso `PerFieldAnalyzerWrapper` viene passato sia all'`IndexWriterConfig` sia al `QueryParser` in `Searcher.java`, garantendo coerenza tra i token generati durante l'indicizzazione e quelli cercati a runtime.

3.3 Interfaccia di Ricerca da Console

La classe `Searcher.java` legge in input da console una stringa che inizia con la keyword `nome` o `contenuto` (gestendo anche il refuso `contentuto` riportato nella traccia), seleziona il campo corrispondente e invoca `QueryParser.parse()` sulla restante porzione della query, supportando nativamente termini singoli, combinazioni booleane e *phrase query* racchiuse tra virgolette `"..."`. Di seguito è riportato un estratto di sessione interattiva:

```
Indice aperto (10000 file).
Scrivi una query (es. 'nome lunar' o 'contenuto "magnetic resonance imaging"'), '
    test' o 'esci':

> nome "lunar module"
Trovati 45 documenti in 190 ms (mostrati top 10):
doc4000: space_apollo11_lunar-module_00001.txt (3.5721455)
doc4001: space_apollo11_lunar-module_00002.txt (3.5721455)
doc4002: space_apollo11_lunar-module_00003.txt (3.5721455)
...
```

4 Valutazione Sperimentale

4.1 Tempi di Indicizzazione al Variare della Dimensione del Corpus

Per analizzare come scalano le prestazioni di indicizzazione, in `Indexer.java` ho misurato il tempo richiesto per creare l'indice da zero (comprensivo di lettura dei file da disco e `writer.commit()` finale) su tagli crescenti da 100 a 10.000 documenti sul mio laptop (Windows 11, SSD NVMe con file system NTFS).

File Indicizzati (N)	Tempo Totale (ms)	Media (ms/file)	Throughput (file/s)
100	2.066 ms	20,66 ms	48,4 file/s
500	6.514 ms	13,03 ms	76,8 file/s
1.000	7.921 ms	7,92 ms	126,2 file/s
2.500	12.558 ms	5,02 ms	199,1 file/s
5.000	26.589 ms	5,32 ms	188,0 file/s
10.000	**37.003 ms**	**3,70 ms**	**270,2 file/s**

Tabella 1: Tempi di indicizzazione misurati da 100 a 10.000 file `.txt`.

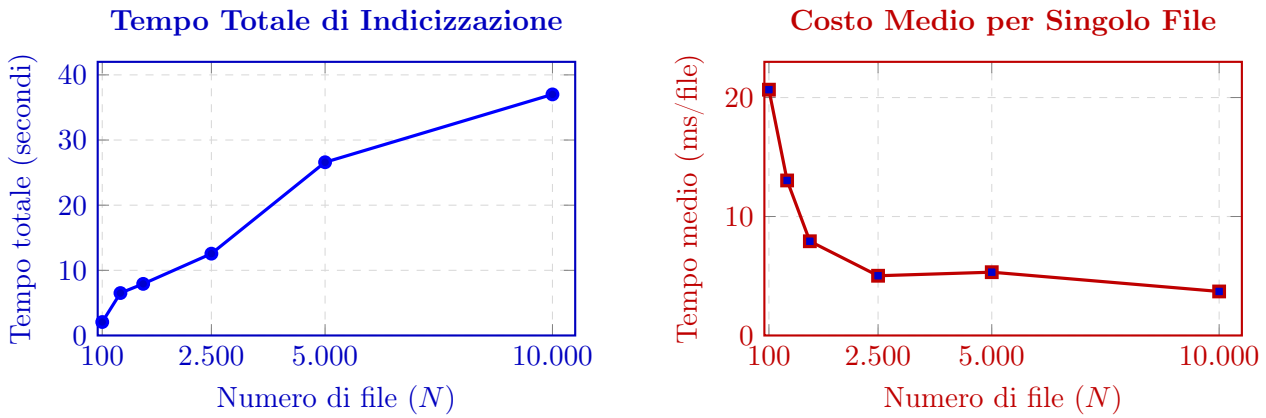


Figura 1: Andamento del tempo totale (a sinistra) e abbattimento del tempo medio per file (a destra) al crescere della dimensione del corpus indicizzato.

Dai dati in Tabella 1 e dai grafici in Figura 1 si nota chiaramente che il costo medio per singolo file diminuisce all'aumentare del numero di documenti: passando da 100 a 10.000 file (100×), il tempo totale cresce di circa 18× (da 2, 07 s a 37, 00 s), mentre il tempo medio per file scende da 20, 66 ms a 3, 70 ms (con un throughput che sale da 48, 4 a 270, 2 file/s).

Questo effetto è dovuto al fatto che su 100 file pesano i costi fissi di inizializzazione dell'`IndexWriter` e di warm-up della JVM, mentre su volumi maggiori Lucene sfrutta i buffer in RAM di `DocumentsWriter` (`DWPT`), scaricando i segmenti su disco (*flush*) solo periodicamente ed eseguendo un unico `commit()` al termine del ciclo. Su Windows, buona parte dei 37 secondi complessivi per 10.000 file è legata all'overhead delle system call su file system NTFS per l'apertura sequenziale di 10.000 piccoli file distinti.

4.2 Valutazione delle 10 Query di Test (Precision, Recall e F1-Score)

Sull'indice completo da **10.000 documenti** ho eseguito 10 query di test tramite il comando `Searcher -test`. Per ogni query sono stati misurati i veri positivi (TP), i falsi positivi (FP), i falsi negativi (FN) e le tre metriche standard:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

ID	Query Console	$\lvert R_q \rvert$ (GT)	Trovati	TP	FP	FN	Precision	Recall	F1	Tempo
Q1	`nome lunar`	60	60	60	0	0	1.000	1.000	1.000	41,08 ms
Q2	`nome "lunar module"`	45	45	45	0	0	1.000	1.000	1.000	15,21 ms
Q3	`nome rtx 4090`	50	70	50	20	0	0.714	1.000	0.833	15,18 ms
Q4	`nome +rtx +4090`	50	50	50	0	0	1.000	1.000	1.000	3,14 ms
Q5	`contenuto vaccinations`	80	80	80	0	0	1.000	1.000	1.000	7,22 ms
Q6	`contenuto magnetic resonance imaging`	40	594	40	554	0	0.067	1.000	0.126	4,20 ms
Q7	`contenuto "magnetic resonance imaging"`	40	34	34	0	6	1.000	0.850	0.919	3,13 ms
Q8	`contenuto "magnetic resonance imaging"~2`	40	40	40	0	0	1.000	1.000	1.000	6,12 ms
Q9	`contenuto "retrieval of information"`	45	45	45	0	0	1.000	1.000	1.000	1,71 ms
Q10	`contenuto +ballistic +missile -nuclear`	50	45	45	0	5	1.000	0.900	0.947	5,95 ms
Media	*Macro-Average sulle 10 query*	–	–	–	–	–	**0.878**	**0.975**	**0.883**	**10,29 ms**

Tabella 2: Risultati sperimentali delle 10 query sull'indice da 10.000 documenti.

4.3 Analisi dei Risultati Query per Query

- **Q1 (`nome lunar`) e Q2 (`nome "lunar module"`):** mettono alla prova `wordDelimiterGraph` sui nomi dei file. Con **Q1** vengono trovati tutti i 60 file spaziali lunari (45 file `space_apollo11_lunar-module_*.txt` e 15 file `space_lunar_rover_command_module_*.txt`). Passando alla *phrase query* **Q2** tra virgolette, Lucene verifica l'adiacenza delle posizioni dei token

`lunar` e `module`: i 15 file del *Lunar Rover* vengono scartati (perché tra `lunar` e `module` ci sono `rover` e `command`) e vengono restituiti esattamente i 45 file del modulo lunare ($P = 1.000, R = 1.000$).

- **Q3 (`nome rtx 4090`) vs Q4 (`nome +rtx +4090`):** verificano lo split tra lettere e numeri (`rtx4090` → `rtx`, `4090`) e il comportamento di default del `QueryParser`. In **Q3**, i termini senza operatori vengono cercati in `OR` (`SHOULD`): oltre ai 50 file `rtx4090` (che compaiono comunque ai primi 50 posti della classifica grazie allo score `BM25` più alto), vengono estratti anche i 20 file distrattori `rtx3080` che contengono il token `rtx`, abbassando la Precision a $50/70 = 0.714$. Aggiungendo l'operatore obbligatorio `+` (`MUST`) in **Q4**, i 20 falsi positivi vengono eliminati e la Precision torna a 1.000.

- **Q5 (`contenuto vaccinations`):** dimostra il vantaggio dello stemming sul campo `contenuto`. Gli 80 file medici rilevanti contengono le parole *vaccinations* (30 file), *vaccinated* (25 file) e *vaccinating* (25 file). Poiché `EnglishAnalyzer` riduce sia la query sia i tre termini allo stesso stem `vaccin`, la query recupera tutti gli 80 documenti ($R = 1.000$). Senza stemming avremmo recuperato solo i 30 file con la parola esatta *vaccinations* ($R = 0.375$).

- **Q6, Q7 e Q8 (Confronto tra query libera, Phrase Query e Slop):** questo gruppo di query mostra chiaramente il trade-off tra Precision e Recall su un corpus ampio:

  - in **Q6** (`contenuto magnetic resonance imaging`), i tre termini sono cercati in `OR`. Inoltre, lo stemmer trasforma `imaging` nella radice `imag`, che fa match anche con le parole *image* e *images* presenti in centinaia di articoli della categoria grafica (`comp.graphics`). Di conseguenza, Lucene recupera tutti i 40 file medici rilevanti ($R = 1.000$), ma restituisce anche **554 falsi positivi** (35 file spaziali con *magnetic/resonance* + 519 file di computer graphics), facendo scendere la Precision a 0.067;
  - in **Q7** (`contenuto "magnetic resonance imaging"`), le virgolette forzano una `PhraseQuery` esatta (`slop = 0`): i 554 falsi positivi spariscono subito ($P = 1.000$), ma la Recall scende a 0.850 (34/40) perché vengono persi 6 articoli medici rilevanti che contengono la variante *"magnetic resonance functional imaging"*;
  - in **Q8** (`contenuto "magnetic resonance imaging"~2`), impostando uno `slop` di 2 posizioni si recuperano anche i 6 documenti con l'aggettivo *functional* nel mezzo senza reintrodurre falsi positivi, ottenendo $P = 1.000, R = 1.000, F_1 = 1.000$.

- **Q9 (`contenuto "retrieval of information"`):** mostra l'effetto combinato di rimozione delle stopword e stemming dentro una *phrase query*. Sia i 30 file con *"retrieval of information"* sia i 15 file con *"retrieving the information"* vengono indicizzati con la stessa sequenza posizionale (`retriev` → *stopword* → `inform`) e vengono tutti recuperati (45/45), mentre i 5 file distrattori con *"retrieval and archival storage of confidential information"* vengono esclusi perché i due token si trovano a una distanza maggiore.

- **Q10 (`contenuto +ballistic +missile -nuclear`):** combina due clausole obbligatorie (`+`) e una proibita (`-`, `MUST_NOT`). Il filtro `-nuclear` scarta correttamente tutti i 30 articoli sulle testate nucleari mantenendo la Precision a 1.000, ma causa 5 falsi negativi ($R = 45/50 = 0.900$) perché esclude anche 5 articoli rilevanti sui missili convenzionali in cui compare il termine composto *"non-nuclear"* (che il tokenizer spezza in `non` e `nuclear`). È un esempio concreto di come l'operatore `MUST_NOT` aumenti la Precision ma possa ridurre leggermente la Recall quando un termine vietato viene usato in senso negato nel documento.